DAA

MGU BCA - 3RD SEM

Design and Analysis of Algorithms

Complete Important Questions

By Aben Sojan

Exam - 12 Nov 2026

MODULE 1

Introduction and Analysis

1

EXAM PATTERN (from real MGU question papers)

DAA Honours exam: 50 marks, 1 hour 30 minutes. Part A: 10 x 2 marks, all compulsory, short questions from all modules. Part B: 3 x 5 marks, focused explanations with examples. Part C: 1 x 15 marks, full algorithm trace or essay. Total: 20 + 15 + 15 = 50. Part C clusters on MST (Prim's/Kruskal's), knapsack, Floyd-Warshall, N-Queens.

Every module, every important, repeated and sure-mark question for Design and Analysis of Algorithms, each with a full-mark model answer.

1 Define an algorithm. What are its characteristics?

SURE MARK (May 2019, Oct 2025 H)

An algorithm is a finite sequence of well-defined, unambiguous steps that solves a problem or performs a computation.

It takes zero or more inputs and produces at least one output, with a clear relationship between them.

Characteristic 1, Input: zero or more quantities supplied to the algorithm from outside.

Characteristic 2, Output: at least one quantity produced as the result.

Characteristic 3, Definiteness: every step must be precisely defined, leaving no room for guessing.

Characteristic 4, Finiteness: the algorithm must terminate after a finite number of steps for every input.

Characteristic 5, Effectiveness: every step must be basic enough to be carried out in principle, even with pencil and paper.

Example: adding two numbers - read a, read b, compute c = a + b, print c.

Each step is clear, it always stops, and it always gives the right sum.

In short: an algorithm is a correct, clear and terminating recipe for solving a problem.

2 What are the different algorithm design techniques? Name them.

REPEATED (Oct 2021, Feb 2025)

Algorithm design techniques are general strategies for building algorithms; this syllabus uses four of them.

Divide and conquer: split the problem into smaller subproblems of the same type, solve them recursively, and combine the answers.

Example: merge sort.

Greedy: build the answer step by step, always taking the locally best choice and never reconsidering it.

Example: Dijkstra's shortest path.

Dynamic programming: solve overlapping subproblems once, store the answers and reuse them instead of recomputing.

Example: Floyd Warshall all-pairs shortest path.

Backtracking: build candidates step by step and abandon a partial candidate as soon as it cannot lead to a valid answer.

Example: N-Queens.

Choosing the right technique is the first step of algorithm design; Modules 2 and 3 study each one in depth.

3 What is the difference between an algorithm and a program?

IMPORTANT

An algorithm is the idea; a program is that idea written down in a programming language so a machine can run it.

An algorithm is described in plain language or pseudocode and is machine independent, while a program is written in Python, C or Java for a real computer.

The same algorithm can become many different programs in different languages.

An algorithm must always terminate (finiteness is compulsory), but a program need not - an operating system is meant to run forever.

An algorithm is analysed for correctness and time/space complexity; a program is additionally tested, debugged and maintained.

Example: binary search written as pseudocode is the algorithm; the same logic typed into Python is the program.

In short: every program implements some algorithm, but an algorithm exists even before any code is written.

4 Differentiate space complexity and time complexity of algorithms with an example.

SURE MARK (May 2019, Oct 2021, Feb 2025)

Time complexity is the number of basic steps an algorithm takes, expressed as a function of the input size n.

Space complexity is the amount of memory an algorithm needs, also expressed as a function of n.

Time is counted in operations such as comparisons and assignments; space is counted in variables, arrays and call-stack frames.

Both are written with asymptotic notation so that we compare growth rates, not exact counts.

Example: iterative binary search takes O(log n) time and O(1) extra space, but the recursive version needs O(log n) stack space.

Example: merge sort takes O(n log n) time and O(n) extra space for the temporary merge arrays.

There is often a trade-off: using more memory, like a lookup table, can buy a faster running time.

In short: time is how long it runs, space is how much memory it needs - analyse both before choosing an algorithm.

5 Define space complexity. What are its components?

IMPORTANT

Space complexity is the amount of memory an algorithm needs, expressed as a function of the input size n.

It has two components.

Fixed space: memory for the program code itself and for constants; it does not change with input size.

Variable space: memory that grows with the input - space for the input data, for extra variables and data structures, and for the recursion call stack.

Example: adding n numbers stored in an array needs O(n) space for the array plus O(1) for the loop variables.

Recursive algorithms add one stack frame per active call, so recursion depth directly adds to variable space.

We usually count only the extra workspace beyond the input, and we express it with Big-O, e.g. O(1) or O(n).

A good algorithm balances time and space: merge sort is fast but needs O(n) extra space, while

quick sort sorts in place.

6 Why does recursion increase space complexity?

IMPORTANT

Every active function call needs memory for its local variables, parameters and return address; this memory lives in a stack frame on the call stack.

A recursive call cannot finish until the calls it made finish, so all frames from the first call down to the deepest one stay on the stack at the same time.

Therefore a recursion of depth d needs O(d) extra stack space on top of everything else the algorithm uses.

Example: recursive factorial(5) keeps 5 frames alive at the deepest point, so it uses O(n) stack space for input n.

The iterative version of the same loop uses only O(1) extra space, since no calls pile up.

This is also why very deep recursion crashes with a stack overflow: the stack memory runs out.

In short: recursion trades elegance for stack memory, and depth directly becomes space.

7 Define time complexity. Explain best, average and worst cases.

SURE MARK (May 2019, Oct 2021, Feb 2025)

Time complexity measures how the running time of an algorithm grows when the input size n grows.

We count basic operations (comparisons, assignments) instead of seconds, so the measure is machine independent.

Best case: the minimum time over all inputs of size n.

Example: linear search finding the key at the first position takes O(1).

Average case: the expected time over typical inputs; it needs assumptions about how inputs are distributed.

Worst case: the maximum time over all inputs of size n.

Example: linear search scanning all n items takes O(n).

The worst case is used most often because it gives a guarantee - the algorithm will never be slower than this.

For linear search on n items: best O(1), average O(n/2) which is O(n), worst O(n).

Reporting all three cases gives the complete picture, but the worst case is the standard promise of an algorithm.

8 What is performance analysis of an algorithm? Explain.

REPEATED (May 2019, Oct 2021)

Performance analysis of an algorithm means measuring how its running time and memory use grow as the input size n grows.

It has two parts: time complexity (the number of steps) and space complexity (the memory used).

Analysis is done theoretically by counting basic operations, not by timing on a machine, so the result is machine independent.

Growth is expressed with asymptotic notation such as Big-O; for example linear search is O(n) time and O(1) extra space.

Example: for sorting n numbers, bubble sort is O(n^2) while merge sort is O(n log n) - analysis tells us merge sort wins for large n without running either.

We usually analyse the worst case, because it guarantees how the algorithm behaves on its hardest input.

In short: performance analysis lets us compare algorithms on paper before writing any code.

9 Why do we analyse worst-case time complexity?

IMPORTANT

The worst case gives a hard guarantee: no input of size n can make the algorithm slower than this bound.

Users and system designers plan with guarantees - a video player must decode every frame in time, not just the average frame.

The average case needs probability assumptions about inputs, and real inputs are often not random; a wrong assumption makes the analysis useless.

The best case is usually trivial and tells us nothing about how the algorithm behaves on hard inputs.

For many algorithms the worst case is also easy to find and analyse, e.g. quick sort on an already sorted array.

That is why textbooks and interviews quote the worst case by default: it is safe, assumption-free and informative.

10 Define Big-O notation with its mathematical definition.

REPEATED (Feb 2025, Oct 2025 H)

Big-O notation gives an asymptotic UPPER bound: it says a function never grows faster than some simpler function g(n), ignoring constant factors.

Mathematical definition: f(n) = O(g(n)) if there exist positive constants c and n0 such that f(n) <= c * g(n) for all n >= n0.

The constant c absorbs multiplicative factors, and n0 says the bound only needs to hold for large inputs.

Worked example: show 3n + 2 = O(n).

Need 3n + 2 <= c*n for n >= n0.

For n >= 2, 3n + 2 <= 3n + n = 4n, so c = 4 and n0 = 2 work.

Intuition: for large n, 3n + 2 behaves like n up to a constant factor, so it is O(n).

Big-O lets us compare algorithms by growth rate alone: O(n log n) always beats O(n^2) for large n.

11 Define Omega and Theta notations.

REPEATED (Feb 2025, Oct 2025 H)

Omega notation gives an asymptotic LOWER bound: the function grows at least as fast as g(n).

Definition: f(n) = Omega(g(n)) if there exist c > 0 and n0 such that f(n) >= c * g(n) for all n >= n0.

Theta notation gives a TIGHT bound: the function grows at the same rate as g(n), trapped between two constant multiples.

Definition: f(n) = Theta(g(n)) if there exist c1 > 0, c2 > 0 and n0 such that c1*g(n) <= f(n) <= c2*g(n) for all n >= n0.

In other words, Theta means both the O and the Omega bound hold with the same g(n).

Example: f(n) = 5n^2 + 3n is Theta(n^2), because for n >= 1 we have 5n^2 <= f(n) <= 8n^2.

Memory aid: Big-O is at most, Omega is at least, Theta is exactly this fast in growth terms.

12 Show that 2n^2 + 10n is O(n^2).

IMPORTANT

We must find constants c > 0 and n0 such that 2n^2 + 10n <= c*n^2 for all n >= n0.

For n >= 1 we have n <= n^2, so 10n <= 10n^2.

Substituting: 2n^2 + 10n <= 2n^2 + 10n^2 = 12n^2 for all n >= 1.

So choosing c = 12 and n0 = 1 satisfies the Big-O definition.

Hence 2n^2 + 10n = O(n^2), as required.

The constants are not unique: c = 3 and n0 = 10 also work, since 10n <= n^2 whenever n >= 10.

The lower-order term 10n is swallowed by n^2 for large n, which is exactly what Big-O captures.

13 Arrange in increasing order of growth: O(n^2), O(1), O(n log n), O(2^n), O(log n), O(n).

IMPORTANT

Growth rates ordered from slowest to fastest give the standard complexity ladder.

O(1): constant - the time does not change no matter how big n gets.

O(log n): logarithmic - grows very slowly, e.g. binary search halving the input.

O(n): linear - time grows in direct proportion to n, e.g. a single scan.

O(n log n): linearithmic - slightly worse than linear, e.g. merge sort.

O(n^2): quadratic - doubling n quadruples the time, e.g. bubble sort.

O(2^n): exponential - adding 1 to n doubles the time, e.g. naive Fibonacci recursion.

Final increasing order: O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n).

Memory trick: constant, log, linear, linearithmic, quadratic, exponential.

This ladder is the scale every complexity comparison in the subject is measured against.

14 What is the difference between Big-O and Theta?

IMPORTANT

Big-O is only an upper bound: it says the algorithm grows no faster than g(n), but it may grow much slower.

Theta is a tight bound: it pins the growth from both above and below, describing the exact growth rate.

Example: binary search is O(n) - a true but loose statement - and Theta(log n), which is tight and informative.

Saying an algorithm is O(n^2) leaves open whether it is really quadratic or actually linear; Theta(n) would settle the question.

In practice we quote Big-O for worst-case guarantees and Theta when we have proved the exact behaviour.

Prefer Theta whenever you can prove it; Big-O is the safe fallback that is always available.

15 What is recursion? Explain with an example.

IMPORTANT

Recursion is a technique where a function calls itself on a smaller version of the same problem, until it reaches a trivial base case.

Every recursive definition needs two parts: a base case whose answer is known directly, and a recursive case that shrinks the problem toward the base case.

Pseudocode for factorial:

```
factorial(n):
if n == 0: return 1 # base case
return n * factorial(n-1) # recursive case
```

Trace of factorial(3): 3 * factorial(2), which is 3 * 2 * factorial(1), which is 3 * 2 * 1 * factorial(0) = 3 * 2 * 1 * 1 = 6.

Each call waits for the smaller call to finish, then combines the result; the call stack remembers the pending multiplications.

Recursion mirrors the structure of trees, divide-and-conquer and backtracking problems, which is why it appears everywhere in this subject.

16 What happens if a recursive function has no base case?

IMPORTANT

Without a base case the function keeps calling itself on smaller inputs forever; this is infinite recursion.

Every call pushes a new stack frame, so the call stack grows without limit until the program crashes with a stack overflow.

Example: factorial without the n == 0 check would call factorial(-1), factorial(-2), and so on until memory runs out.

Even with a base case present, the recursive step must actually move toward it; calling factorial(n+1) would also never terminate.

So two rules: always include a base case, and always shrink the input on each recursive call.

The base case is the emergency brake of recursion - no brake, no stopping, and the program crashes.

17 What is a recurrence relation? Give an example.

IMPORTANT

A recurrence relation expresses the running time T(n) for input size n in terms of the running time for smaller inputs, plus the cost of the current step.

It always comes with one or more base cases that give T directly for tiny inputs.

Example from merge sort: T(n) = 2T(n/2) + n, with T(1) = 1.

Reading it: sorting n items costs the time to sort two halves (2T(n/2)) plus n for merging them.

Solving the recurrence - by substitution, recursion tree or the Master method - gives the closed form, here T(n) = Theta(n log n).

Recurrences are the standard language for analysing recursive algorithms, and every divide-and-conquer algorithm yields one.

18 Explain the substitution method for solving recurrences.

IMPORTANT

The substitution method solves a recurrence in two steps: guess the form of the answer, then prove the guess correct by mathematical induction.

Step 1, guess: use experience or a recursion tree to propose a bound, e.g. T(n) = O(n log n) for T(n) = 2T(n/2) + n.

Step 2, prove: assume the bound holds for all smaller sizes, i.e. T(k) <= c*k*log(k) for k < n, and show it follows for n.

```
T(n) = 2T(n/2) + n <= 2*(c*(n/2)*log(n/2)) + n = c*n*log(n) - c*n + n <= c*n*log(n) for c >= 1.
```

The induction closes, so the guess is proved and T(n) = O(n log n) is established.

The method needs a good initial guess; drawing a recursion tree first usually supplies it.

Substitution is the rigorous proof technique behind most recurrence solutions expected in exams.

19 Solve T(n) = 2T(n/2) + n using a recursion tree.

IMPORTANT

A recursion tree draws every recursive call as a node labelled with its non-recursive cost, arranged level by level.

Level 0: one node for the whole problem of size n, costing n.

```
Level 1: two nodes of size n/2, each costing n/2, for a level total of n.
```

```
Level 2: four nodes of size n/4, each costing n/4, for a level total of n.
```

In general, level i has 2^i nodes of size n/2^i, each costing n/2^i, so every level costs exactly n.

The tree stops when the subproblem size reaches 1, which happens after log2(n) levels, since n/2^i = 1 gives i = log n.

Total cost = (cost per level) x (number of levels) = n x log n.

Hence T(n) = Theta(n log n); this is the classic merge-sort analysis every student must be able to reproduce.

The tree makes the n log n visible: log n levels, each doing n work.

MODULE 2

Divide and Conquer, Greedy

2

EXAM PATTERN (from real MGU question papers)

DAA Honours exam: 50 marks, 1 hour 30 minutes. Part A: 10 x 2 marks, all compulsory, short questions from all modules. Part B: 3 x 5 marks, focused explanations with examples. Part C: 1 x 15 marks, full algorithm trace or essay. Total: 20 + 15 + 15 = 50. Part C clusters on MST (Prim's/Kruskal's), knapsack, Floyd-Warshall, N-Queens.

1 Explain the divide and conquer strategy with its three steps.

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

Divide and conquer is an algorithm design strategy that solves a problem by breaking it into smaller copies of itself.

Step 1, DIVIDE: split the problem into smaller subproblems, usually of the same type and roughly half the size.

Step 2, CONQUER: solve the subproblems recursively; when a subproblem becomes tiny, solve it directly.

Step 3, COMBINE: merge the subproblem answers into the final answer for the original problem.

Example, merge sort: divide the array into two halves, conquer by sorting each half, combine by merging the two sorted halves.

The running time naturally becomes a recurrence, e.g. T(n) = 2T(n/2) + n, which solves to O(n log n).

Whenever a problem splits into independent smaller copies of itself, divide and conquer is the first strategy to try.

2 What are the advantages of divide and conquer?

IMPORTANT

Better complexity: splitting often drops the running time dramatically, e.g. O(n log n) sorting instead of O(n^2).

Parallelism: the subproblems are independent of each other, so they can be solved simultaneously on multiple processors.

Simple analysis: the recurrence T(n) = aT(n/b) + f(n) is solved mechanically by the Master method.

Cache friendliness: working on small chunks fits better into CPU caches, which speeds up real programs noticeably.

Elegant code: the recursive structure is short and mirrors the problem definition, so it is easy to write correctly.

These advantages explain why merge sort, quick sort, binary search and Strassen's algorithm are all divide-and-conquer algorithms.

3 Write the control abstraction of the divide and conquer method.

IMPORTANT (Oct 2025 H)

The control abstraction is the general recursive skeleton of divide and conquer, independent of any specific problem.

```
Algorithm DAndC(P)
// P is the problem instance, n its size
if Small(P) then return S(P)
else
divide P into P1, P2, ..., Pk, k >= 1
apply DAndC to each subproblem
return Combine(DAndC(P1), DAndC(P2), ..., DAndC(Pk))
```

Small(P) tests whether P is tiny enough to solve directly; S(P) solves such a tiny instance directly.

Divide splits P into k smaller instances of the same type; each is conquered by a recursive call.

Combine merges the subproblem answers into the answer for the original problem P.

Example: in merge sort, Small(P) is true for a one-element array, and Combine is the merge step.

In short: divide, conquer recursively, combine - every divide and conquer algorithm follows this skeleton.

4 Explain binary search with an example. What is its time complexity?

IMPORTANT

Binary search finds a key in a SORTED array by repeatedly throwing away the half that cannot contain the key.

Pseudocode:

```
low = 0; high = n - 1
while low <= high:
mid = (low + high) // 2
if a[mid] == key: return mid
```

```
elif key < a[mid]: high = mid - 1
else: low = mid + 1
return -1 # not found
```

Trace: search 23 in [5, 12, 23, 34, 45]. mid = index 2, a[2] = 23 = key, found immediately.

Longer trace: search 34. mid = 23 < 34, so low = 3; now mid = index 3, a[3] = 34, found.

Each comparison halves the search range, giving the recurrence T(n) = T(n/2) + 1.

Solving it: after k steps the size is n/2^k, which reaches 1 when k = log2(n), so T(n) = O(log n).

Space is O(1) for the loop version, since only a few variables are used.

Halving the problem at every step is what makes binary search exponentially faster than a linear scan.

5 Why must the array be sorted for binary search?

IMPORTANT

The whole trick of binary search is discarding half the array after a single comparison with the middle element.

That decision is valid only if everything left of the middle is smaller and everything right is larger - which is exactly what sorted order guarantees.

In an unsorted array the key could be hiding in the half we throw away, so the algorithm would wrongly report not found.

Therefore sorting is a precondition, not an optimisation: we sort once at O(n log n) and then enjoy O(log n) searches forever after.

Sorted order is the logical foundation the halving argument stands on; without it the algorithm is simply incorrect.

6 How many comparisons does the straightforward max-min algorithm need?

SURE MARK (Oct 2021, Feb 2025, Oct 2025 H)

The straightforward method finds the maximum and the minimum in two separate scans of the array.

First scan: compare each new element with the current maximum - that takes n - 1 comparisons.

Second scan: compare each new element with the current minimum - another n - 1 comparisons.

Total in the worst case: (n - 1) + (n - 1) = 2n - 2 comparisons.

Example: for n = 100 numbers, that is 198 comparisons.

It is simple to write but wasteful, because every element is compared twice - once against the max and once against the min.

The divide-and-conquer version below cuts this to about 3n/2 - 2 by comparing elements with each other first.

7 **Explain the divide and conquer approach for max and min.**

SURE MARK (Oct 2021, Feb 2025, Oct 2025 H)

Idea: instead of comparing every element against the running max and min, first pair elements up and compare them with each other.

Divide the array into two halves; recursively find (max1, min1) of the left half and (max2, min2) of the right half.

```
overall max = larger of max1 and max2 # 1 comparison
overall min = smaller of min1 and min2 # 1 comparison
```

Combining two halves costs only 2 comparisons, giving the recurrence T(n) = 2T(n/2) + 2.

This solves to about 3n/2 - 2 comparisons, clearly better than the straightforward 2n - 2.

Example: for n = 100, divide and conquer needs about 148 comparisons versus 198 for the naive method.

The saving comes from letting elements eliminate each other before facing the current champions.

8 **For an array of 6 elements, compare the number of comparisons needed to find both the maximum and minimum using linear search and divide and conquer.**

SURE MARK (Oct 2021, Feb 2025, Oct 2025 H)

Linear search (straightforward method): compare each new element with the current max and the current min - 2 comparisons per element after the first.

Comparisons = 2n - 2 = 2(6) - 2 = 10 comparisons.

Divide and conquer (pair method): pair up the 6 elements - 3 comparisons produce 3 pair-winners and 3 pair-losers.

Find the max of the 3 winners: 2 comparisons. Find the min of the 3 losers: 2 comparisons.

Total = 3 + 2 + 2 = 7 comparisons.

So divide and conquer needs 7 comparisons against 10 for linear search - 3 fewer for the same answer.

Answer: linear search 10 comparisons, divide and conquer 7 comparisons.

9 Explain merge sort with an example and analyse its complexity.

SURE MARK (May 2019, Oct 2021, Feb 2025)

Merge sort is the classic divide-and-conquer sort: split the array into halves, sort each half recursively, then merge the sorted halves.

Pseudocode:

```
mergesort(a):
if len(a) <= 1: return a
mid = len(a) // 2
left = mergesort(a[:mid]); right = mergesort(a[mid:])
return merge(left, right) # linear scan taking the smaller front element
```

Trace on [38, 27, 43, 3]: split into [38, 27] and [43, 3]; these sort to [27, 38] and [3, 43].

Merge step: compare 27 vs 3, take 3; 27 vs 43, take 27; 38 vs 43, take 38; then 43.

Result: [3, 27, 38, 43].

Recurrence: two halves plus a linear merge gives T(n) = 2T(n/2) + n, which solves to O(n log n).

Because the split is always exactly half, this bound holds for the best, average AND worst case alike.

Space cost: O(n) extra for the merge buffers.

Merge sort is also stable: equal elements keep their original relative order.

Guaranteed O(n log n) in every case is why merge sort is the safe choice for sorting.

10 Why is merge sort O(n log n) even in the worst case?

SURE MARK (May 2019, Oct 2021, Feb 2025)

The split step always cuts the array into two exactly equal halves, no matter what the data looks like.

So the recursion depth is always log2(n); no input can make the tree deeper or shallower.

At every level, the merge step scans all n elements once across the subproblems, doing n total work per level.

Total work = (work per level) x (number of levels) = n x log n, for every possible input without exception.

Contrast quick sort: its split depends on the pivot, so a bad pivot gives an unbalanced tree and O(n^2).

Merge sort's perfectly balanced partition is built into the algorithm, so the worst case cannot escape O(n log n).

11 What are the drawbacks of merge sort?

IMPORTANT

Extra memory: merging needs O(n) auxiliary space, while quick sort and heap sort work in place.

Recursion overhead: for tiny subarrays the function-call cost exceeds the sorting work, making it slower than insertion sort there.

Not adaptive: it does the full n log n work even on an already sorted array, unlike insertion sort's O(n) best case.

In practice libraries fix this with a hybrid: use merge sort down to small sizes, then finish with insertion sort.

So merge sort trades memory for a rock-solid time guarantee - the right call when memory is available.

12 Illustrate the recursion tree of merge sort for 10 elements.

IMPORTANT (Oct 2021)

The recursion tree of merge sort shows how the array keeps splitting until single elements, then merges back up.

Take [38, 27, 43, 3, 9, 82, 10, 5, 21, 16] (10 elements).

Level 0: one node holding all 10 elements.

Level 1: two nodes of 5: [38, 27, 43, 3, 9] and [82, 10, 5, 21, 16].

Level 2: four nodes of sizes 3, 2, 3, 2: [38, 27, 43], [3, 9], [82, 10, 5], [21, 16].

Level 3: [38, 27], [43], [3], [9] and [82, 10], [5], [21], [16].

Level 4: ten single-element leaves - the base case, each already sorted.

Merging climbs back up: every level merges n elements in total, and there are ceil(log2 10) = 4 levels, so the total work is O(n log n).

In short: the tree has 4 levels above the leaves and shows exactly why merge sort costs O(n log n).

13 Explain quick sort partitioning with an example.

SURE MARK (May 2019, Oct 2021, Feb 2025)

Quick sort picks one element as the pivot, partitions the array so smaller elements go left and larger go right, then sorts both sides recursively.

Lomuto partition pseudocode:

```
pivot = a[low]; i = low
for j in range(low+1, high+1):
if a[j] < pivot: i += 1; swap(a[i], a[j])
swap(a[low], a[i]) # pivot into its final position; return i
```

Trace on [10, 7, 8, 9, 1, 5] with pivot 10: every smaller element is swept left, giving [7, 8, 9, 1, 5, 10].

The pivot 10 is now fixed at the end and never moves again; recurse on [7, 8, 9, 1, 5].

Each partition scans its segment once, costing O(n) for the combine step of that level.

Since every element becomes a pivot exactly once, each is placed into its final position exactly once.

Partitioning is the heart of quick sort - everything else is just recursion on the two sides.

14 Analyse the best and worst cases of quick sort.

SURE MARK (May 2019, Oct 2021, Feb 2025)

The behaviour of quick sort depends entirely on where the pivot lands after partitioning.

Best case: the pivot always lands in the middle, splitting the array into two equal halves.

`Recurrence: T(n) = 2T(n/2) + n (partitioning costs n), which solves to O(n log n).`

Worst case: the pivot is always the smallest or largest element, so one side is empty and the other holds n-1 elements.

`Recurrence: T(n) = T(n-1) + n = n + (n-1) + ... + 1 = O(n^2).`

This worst case strikes with an already sorted array when the first element is always chosen as pivot.

The average case over random inputs is O(n log n), which is why quick sort is fast in practice despite the bad worst case.

In short: balanced pivots give n log n, degenerate pivots give n^2 - pivot choice is everything.

15 Compare merge sort and quick sort.

REPEATED (Oct 2021, Feb 2025)

Both are divide-and-conquer sorting algorithms with O(n log n) average time, but they differ in guarantees and resources.

Time: merge sort is O(n log n) in best, average and worst case; quick sort is O(n log n) on average but O(n^2) in the worst case.

Space: merge sort needs O(n) extra memory for merging; quick sort sorts in place using only O(log n) recursion stack.

Stability: merge sort is stable (equal elements keep their order); quick sort's swapping makes it unstable.

Practical speed: quick sort usually wins because of smaller constants and better cache behaviour from in-place access.

Choice rule: need a guaranteed time bound or stability - merge sort; need speed and low memory - quick sort.

This comparison is a favourite exam question because it tests the whole divide-and-conquer chapter at once.

16 How can the quick sort worst case be avoided?

IMPORTANT

The worst case comes from bad pivots, so the fix is to choose pivots that are unlikely to be extreme.

Random pivot: pick the pivot randomly instead of always the first element; an adversary cannot then feed a killer input, and the expected time stays O(n log n).

Median-of-three: take the median of the first, middle and last elements as the pivot; this kills the sorted-input worst case completely.

Hybrid approach: switch to insertion sort for tiny subarrays (e.g. below 10 elements) to cut recursion overhead.

Introspective sort, used in the C++ STL, goes further: it watches the recursion depth and switches to heap sort if it grows too deep, guaranteeing O(n log n).

With a decent pivot rule, the O(n^2) case becomes a theoretical curiosity rather than a practical danger.

17 Explain Strassen's matrix multiplication. Why is it faster?

SURE MARK (Oct 2021, Feb 2025, Oct 2025 H)

Standard matrix multiplication of two n x n matrices needs O(n^3) operations; a plain divide-and-conquer version needs 8 half-size multiplications.

Strassen's insight: with clever additions and subtractions of the submatrices, only 7 multiplications (M1 to M7) are enough to build all four result blocks.

One expensive multiplication is traded for several cheap additions, and multiplication dominates the cost.

Recurrence: 7 subproblems of size n/2 plus O(n^2) additions gives T(n) = 7T(n/2) + O(n^2).

By the Master method this solves to O(n^2.81), since log2(7) is about 2.81, which beats n^3 for large n.

Example: for n = 1024, n^3 is about 10^9 operations while n^2.81 is roughly 3 x 10^8 - a 3x saving.

Strassen proved that matrix multiplication is fundamentally easier than cubic, opening the whole field of fast matrix algorithms.

18 Write the recurrence for Strassen's algorithm and solve it.

SURE MARK (Oct 2021, Feb 2025, Oct 2025 H)

At each level Strassen creates 7 subproblems of half the size and does O(n^2) additions and subtractions to combine them.

`Recurrence: T(n) = 7T(n/2) + c*n^2, with T(1) = O(1).`

Apply the Master method: a = 7, b = 2, f(n) = n^2.

Compare n^(log_b a) = n^(log2 7) = n^2.81 against f(n) = n^2.

Since n^2.81 grows polynomially faster than n^2, we are in Master case 1: the subproblems dominate the cost.

Therefore T(n) = Theta(n^(log2 7)) = Theta(n^2.81).

The exponent drops from 3 to 2.81 purely because 7 multiplications replaced 8.

19 What are the limitations of Strassen's algorithm?

IMPORTANT

High constant factors: the many additions, subtractions and complex bookkeeping make each recursion level expensive.

Crossover point: it only beats the naive O(n^3) method for large matrices, typically n above a few hundred.

Extra memory: the intermediate sum and difference matrices need additional storage beyond the inputs.

Numerical stability: more additions mean more rounding-error accumulation in floating-point arithmetic.

Implementation complexity: the code is far harder to write and debug than three nested loops.

So Strassen is a theoretical breakthrough that libraries use selectively, not a drop-in replacement for everyday multiplication.

20 Explain the greedy strategy. When does it work?

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

The greedy strategy builds an answer step by step, always taking the choice that looks best right now, and never reconsidering earlier choices.

It is simple and fast, but it only works for problems with two structural properties.

Greedy-choice property: a globally optimal answer can be assembled from locally optimal choices - the first greedy pick is always part of some optimum.

Optimal substructure: after making the greedy choice, the remaining problem is a smaller copy of the original.

Example where it works: coin change with coins 25, 10, 5, 1 - repeatedly taking the largest coin that fits always uses the fewest coins.

Syllabus examples that are all greedy: fractional knapsack, Prim's, Kruskal's and Dijkstra's algorithms.

Greedy is take the best now, never look back - correct only when the problem's structure forgives short-sightedness.

21 Define feasible solution and optimal solution.

REPEATED (Oct 2021, Feb 2025)

A feasible solution is any candidate answer that satisfies all the constraints of the problem, whether or not it is the best one.

An optimal solution is the feasible solution with the best objective value - maximum profit or minimum cost.

Example, knapsack with capacity 10: item A (weight 6, profit 12) and item B (weight 5, profit 9).

Picking only A is feasible (weight 6 <= 10) with profit 12; picking only B is feasible with profit 9.

Picking A + B together is not feasible (weight 11 > 10).

The optimal solution is picking A alone, profit 12 - the best among all feasible picks.

In short: feasible means allowed, optimal means allowed and best.

22 Give an example where the greedy method fails.

IMPORTANT

The classic failure is the 0/1 knapsack, where items cannot be split.

Items: A (profit 60, weight 10), B (100, 20), C (120, 30); capacity 50.

Greedy by profit-to-weight ratio picks A first (ratio 6), then B (ratio 5): total weight 30, profit 160, and C no longer fits.

The true optimum takes B and C: weight 50, profit 220, which greedy never finds because it committed to A.

The failure happens because the greedy-choice property is missing: the locally best item A is not part of the global optimum.

Note the contrast: fractional knapsack allows taking part of C, and there greedy is optimal.

This example is the standard exam proof that greedy needs the right problem structure.

23 Write and explain the control abstraction of the greedy method.

IMPORTANT (Feb 2025)

The control abstraction is the general skeleton that every greedy algorithm follows.

```
Algorithm Greedy(a, n)
// a[1:n] holds the n inputs
solution = empty
for i = 1 to n do
x = Select(a)
if Feasible(solution, x) then
```

```
solution = Union(solution, x)
return solution
```

Select(a) picks the remaining input that looks best by the greedy criterion, e.g. the highest profit per weight.

Feasible(solution, x) checks that adding x keeps a valid partial answer, e.g. the total weight stays within capacity.

Union(solution, x) adds x to the answer; a greedy choice is never reconsidered later.

Example: fractional knapsack - Select picks the highest-ratio item, Feasible checks the capacity, Union adds it or a fraction of it.

In short: repeatedly select the best-looking feasible input - simple and fast, but correct only when the greedy-choice property holds.

24 What is subset paradigm?

IMPORTANT (Oct 2021)

Greedy problems fall into two paradigms: the subset paradigm and the ordering paradigm.

Subset paradigm: the answer is a subset of the n inputs - we decide for each input whether to take it or not.

Examples: knapsack (which items to pack), Prim's and Kruskal's (which edges form the minimum spanning tree).

Ordering paradigm: the answer is an ordering of the inputs, e.g. scheduling jobs in some sequence.

All the greedy problems in this syllabus - knapsack, Prim's, Kruskal's, Dijkstra's - are subset paradigm: each step picks one more input into the solution set.

In short: in the subset paradigm the answer is built as a set of chosen inputs.

25 Solve fractional knapsack with the greedy method. Give steps.

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

In the fractional knapsack, items may be split, and the goal is maximum profit within capacity M.

Step 1: compute the profit-to-weight ratio p/w for every item.

Step 2: sort items by ratio in descending order - this costs O(n log n).

Step 3: walk down the sorted list; take each whole item if it fits, otherwise take exactly the fraction that fills the knapsack, then stop.

Worked example: M = 50; items A (60, 10), B (100, 20), C (120, 30); ratios 6, 5, 4.

```
Take all of A: weight 10, profit 60. Remaining capacity 40.
```

```
Take all of B: weight 20, profit 100. Remaining capacity 20.
Take 20/30 of C: profit (20/30)*120 = 80. Knapsack full.
```

Total profit = 60 + 100 + 80 = 240, and this is provably optimal.

Optimality argument: any optimum can be rearranged to take items in ratio order without losing profit, so greedy matches it.

Complexity: O(n log n) for sorting plus O(n) for the scan.

Divisibility is what makes the greedy choice safe here, unlike the 0/1 version.

26 **Differentiate fractional and 0/1 knapsack.**

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

The difference is divisibility: fractional knapsack allows taking part of an item; 0/1 knapsack forces take-it-or-leave-it.

Fractional: the greedy algorithm sorting by profit-to-weight ratio is optimal, running in O(n log n).

0/1: greedy can fail (the 60/100/120 example), so dynamic programming over capacity is needed, costing O(n*M).

Fractional is solvable in polynomial time; 0/1 knapsack is NP-hard in general.

Example: with capacity 50 and items (60,10), (100,20), (120,30), fractional gives 240 while 0/1 gives 220 by a different choice.

Memory aid: fractional means greedy works; zero-one means DP is needed.

One word - divisible or not - decides the entire algorithm.

27 **Solve the fractional knapsack instance with weights (10, 20, 30, 25, 28), profits (60, 100, 120, 75, 140) and capacity 50 using the greedy approach.**

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

First compute the profit-to-weight ratio of each item.

Item 1: 60/10 = 6.0; Item 2: 100/20 = 5.0; Item 3: 120/30 = 4.0; Item 4: 75/25 = 3.0; Item 5: 140/28 = 5.0.

Sort by ratio in descending order: Item 1 (6.0), Item 2 (5.0), Item 5 (5.0), Item 3 (4.0), Item 4 (3.0).

Capacity 50. Take Item 1 fully: weight 10, profit 60, remaining capacity 40.

Take Item 2 fully: weight 20, profit 100, remaining capacity 20.

Item 5 weighs 28 but only 20 fits: take the fraction 20/28, profit 140 x 20/28 = 100, remaining capacity 0.

Total profit = 60 + 100 + 100 = 260.

Answer: maximum profit 260, by taking items 1 and 2 fully and 20/28 of item 5.

28 **Explain Prim's algorithm step by step with an example.**

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

Prim's algorithm finds the Minimum Spanning Tree: the cheapest set of edges connecting all vertices without forming cycles.

It grows a single tree, starting from any vertex, always attaching the cheapest edge that reaches outside the tree.

Pseudocode:

```
tree = {start}; repeat V-1 times:
pick the minimum-weight edge (u, v) with u in tree, v outside
add v and the edge to the tree
```

Trace on triangle A-B 1, B-C 2, A-C 3, starting at A.

```
Tree = {A}. Edges from the tree outward: A-B (1), A-C (3). Pick A-B.
Tree = {A, B}. Edges outward: A-C (3), B-C (2). Pick B-C.
Tree = {A, B, C}. Done. MST edges: A-B, B-C with total cost 3.
```

Note A-C (3) was never taken: it would close the cycle A-B-C-A.

Complexity: O(V^2) with a simple array scanning for the minimum; O(E log V) with a min-heap.

The greedy choice is safe by the cut property: the lightest edge crossing any cut belongs to some MST.

Prim's is Dijkstra-like growth, but minimising edge weight instead of distance from the source.

29 **Why does the greedy choice work in Prim's algorithm?**

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

The justification is the cut property of minimum spanning trees.

Cut property: for any partition of the vertices into two sets, the lightest edge crossing between them belongs to at least one MST.

Proof sketch: if some MST avoided that lightest edge, adding it creates a cycle; removing the heaviest edge of that cycle gives another spanning tree that is no costlier - so the lightest edge is safe.

At every step, Prim's current tree defines exactly such a cut: vertices inside the tree versus outside.

The algorithm picks the lightest edge crossing this cut, which the property guarantees is safe to include.

Since every added edge is safe, the finished tree of V-1 edges must be a minimum spanning tree.

The cut property is the one-line proof that Prim's greedy choices never go wrong.

30 Explain Kruskal's algorithm step by step with an example.

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

Kruskal's finds the MST by considering all edges globally, cheapest first, skipping any edge that would form a cycle.

Pseudocode:

```
sort all edges by weight ascending
make a separate set for each vertex
for each edge (u, v) in sorted order:
if find(u) != find(v): add edge to MST; union(u, v)
else: skip it # would create a cycle
```

Trace on edges AB 1, BC 2, AC 3.

```
Sorted: AB(1), BC(2), AC(3). Take AB: sets become {A,B}, {C}.
Take BC: find(B) != find(C), so merge into {A,B,C}. MST so far: AB, BC.
AC: find(A) == find(C), both in {A,B,C} - skip, it would close a cycle.
```

MST cost = 1 + 2 = 3.

Cycle detection uses disjoint-set union (union-find), nearly O(1) per operation with path compression and union by rank.

Complexity is dominated by sorting: O(E log E).

Sort globally, take greedily, skip cycles - that is the whole algorithm.

31 Compare Prim's and Kruskal's algorithms.

IMPORTANT

Both are greedy algorithms that produce a minimum spanning tree of the same minimum total cost.

Growth style: Prim's grows a single tree outward from a start vertex; Kruskal's merges a forest by scanning a globally sorted edge list.

Data needed: Prim's works from the adjacency structure of the graph; Kruskal's works naturally from a plain edge list.

Complexity: Prim's is O(V^2) in the simple version or O(E log V) with a heap; Kruskal's is O(E log E), dominated by sorting.

Dense graphs (many edges): Prim's O(V^2) wins.

Sparse graphs (few edges): Kruskal's O(E log E) wins.

Implementation: Prim's needs a priority queue of frontier vertices; Kruskal's needs union-find for cycle checks.

Memory aid: Prim grows, Kruskal sorts.

Same answer, different strategy - pick based on how dense the graph is.

32 Explain Dijkstra's algorithm step by step with an example.

SURE MARK (May 2019, Oct 2021, Oct 2025 H)

Dijkstra's algorithm finds the shortest paths from a single source to all vertices, requiring non-negative edge weights.

It maintains dist[], the best known distance to each vertex, and repeatedly finalises the closest unsettled vertex.

Pseudocode:

```
dist[source] = 0; dist[others] = infinity; settled = empty
repeat V times:
u = unsettled vertex with smallest dist; mark u settled
for each neighbour v of u: dist[v] = min(dist[v], dist[u] + w(u,v)) #
relaxation
```

Trace: edges S-A 4, S-B 2, B-A 1.

Start: dist = {S:0, A:inf, B:inf}.

```
Settle S (0): relax neighbours -> A = 4, B = 2.
Settle B (2): relax A -> min(4, 2+1) = 3.
Settle A (3). Final distances: S = 0, B = 2, A = 3. The path S-B-A (2+1)
beats the direct S-A (4).
```

Complexity: O(V^2) with a simple array; O(E log V) with a min-heap, since each edge is relaxed once.

Correctness idea: the unsettled vertex with smallest dist cannot be improved later, because all weights are non-negative.

Dijkstra is the standard single-source shortest path algorithm wherever weights are non-negative.

33 Why does Dijkstra's algorithm fail with negative weights?

IMPORTANT

Dijkstra's core promise is: once a vertex is settled, its distance is final and will never improve.

That promise holds only because non-negative weights mean any alternative route through unsettled vertices can only be longer.

A negative edge breaks it: a shorter path to an already settled vertex can appear later via that edge.

Example: S-A weight 5, S-B weight 3, B-A weight -4.

Dijkstra settles A at 5, then discovers S-B-A = -1, but A is already closed.

The true shortest distance to A is -1, which Dijkstra misses - the answer is wrong, not just slow.

Fix: use Bellman-Ford, which relaxes all edges V-1 times and handles negative weights correctly (though not negative cycles).

Settled means final is true only when no edge can pull a distance down afterwards.

34 What is edge relaxation?

IMPORTANT

Relaxation is the single operation at the heart of Dijkstra, Bellman-Ford and Floyd Warshall.

It asks: is the path to v better when routed through u?

```
if dist[u] + w(u, v) < dist[v]: dist[v] = dist[u] + w(u, v)
```

In words: if going source -> ... -> u -> v beats the current best distance to v, update it.

Repeated relaxation propagates improvements outward until no distance can shrink further.

Example: dist[A] = 4, and edge B-A has weight 1 with dist[B] = 2.

Since 2 + 1 = 3 < 4, dist[A] becomes 3.

Every shortest-path algorithm is just relaxation applied in a clever order.

MODULE 3

Dynamic Programming, Graphs, Backtracking

3

EXAM PATTERN (from real MGU question papers)

DAA Honours exam: 50 marks, 1 hour 30 minutes. Part A: 10 x 2 marks, all compulsory, short questions from all modules. Part B: 3 x 5 marks, focused explanations with examples. Part C: 1 x 15 marks, full algorithm trace or essay. Total: 20 + 15 + 15 = 50. Part C clusters on MST (Prim's/Kruskal's), knapsack, Floyd-Warshall, N-Queens.

1 Explain dynamic programming. How is it different from divide and conquer?

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

Dynamic programming solves problems whose subproblems overlap: each distinct subproblem is solved once, stored, and reused instead of being recomputed.

It needs two properties: overlapping subproblems and optimal substructure (an optimum built from optima of subproblems).

Classic example: Fibonacci. fib(5) needs fib(4) and fib(3); fib(4) needs fib(3) again - the same fib(3) is recomputed many times.

Naive recursion explodes to O(2^n); DP stores each fib(k) once, dropping the time to O(n).

Difference from divide and conquer: D&C is for INDEPENDENT subproblems solved separately (merge sort's halves never repeat); DP is for OVERLAPPING subproblems solved once and shared.

Merge sort has no overlap, so DP would not help it; Fibonacci is almost all overlap, so D&C-style recursion is disastrous there.

Two implementation styles: memoization (top-down recursion plus a cache) and tabulation (bottom-up table).

Decision rule: overlap means DP, independent means divide and conquer.

2 Differentiate memoization and tabulation.

IMPORTANT

Both implement dynamic programming; they differ in direction and in which subproblems get computed.

Memoization is top-down: write the natural recursion, but cache every result so a repeated call returns the stored answer.

```
Pseudocode: if n in memo: return memo[n]; memo[n] = solve(n-1) + solve(n-2); return memo[n]
```

Tabulation is bottom-up: fill a table iteratively from the smallest subproblem upward, with no

recursion at all.

```
Pseudocode: table[0] = 0; table[1] = 1; for i in 2..n: table[i] = table[i-1] + table[i-2]
```

Memoization computes only the subproblems actually needed and keeps the recursive structure; tabulation computes everything in order and avoids recursion overhead and stack limits.

Both give O(n) time and O(n) space for Fibonacci.

Memoization is lazier and more intuitive; tabulation is iterative and stack-safe - same DP idea, two flavours.

3 What are the two properties a problem needs for DP?

IMPORTANT

A problem suits dynamic programming only if it has two properties.

Overlapping subproblems: the same smaller problems appear again and again during recursion, so storing answers saves real work.

Without overlap there is nothing to reuse, and plain divide and conquer is the right tool instead.

Optimal substructure: an optimal answer to the whole problem can be constructed from optimal answers to its subproblems.

Without it, combining subproblem answers may not give the global optimum, and DP would produce wrong results.

Example with both: shortest path - the shortest S-to-T path through X combines the shortest S-to-X and X-to-T paths, and subpaths repeat across queries.

Counter-example: merge sort's halves are distinct every time, so it has no overlap and DP adds nothing.

Check both properties before reaching for DP; missing either one means a different technique fits better.

4 Explain the Floyd Warshall algorithm with its recurrence.

SURE MARK (May 2019, Oct 2021, Oct 2025 H)

Floyd Warshall computes the shortest path between EVERY pair of vertices: the all-pairs shortest path problem.

DP idea: let D[k][i][j] be the shortest distance from i to j using only vertices 1..k as intermediate stops.

Recurrence: D[k][i][j] = min(D[k-1][i][j], D[k-1][i][k] + D[k-1][k][j]).

Meaning: the best i-to-j path either ignores vertex k entirely, or goes i -> ... -> k -> ... -> j.

Base: D[0] holds direct edge weights, 0 on the diagonal, infinity where no edge exists.

Pseudocode:

```
initialise D with edge weights
for k in 1..V:
for i in 1..V:
for j in 1..V:
D[i][j] = min(D[i][j], D[i][k] + D[k][j])
```

Only two layers are needed in practice, so the space is O(V^2).

Time is O(V^3) from the three nested loops - the price of computing all pairs.

It handles negative edge weights correctly, but not negative cycles, where distances would keep shrinking forever.

Three loops and one recurrence solve all-pairs shortest paths - the simplest O(V^3) algorithm in the syllabus.

5 Trace Floyd Warshall on a small graph.

SURE MARK (May 2019, Oct 2021, Oct 2025 H)

Graph: 1->2 weight 3, 2->3 weight 2, 1->3 weight 10.

Initial D[0]: row1 [0, 3, 10], row2 [inf, 0, 2], row3 [inf, inf, 0]. k = 1: allow vertex 1 as an intermediate.

No pair improves, since nothing useful routes through vertex 1 yet. k = 2: allow vertex 2.

Check D[1][3]: min(10, D[1][1][2] + D[1][2][3]) = min(10, 3 + 2) = 5.

The path 1->2->3 beats the direct edge. k = 3: allow vertex 3.

No further improvement is possible.

Final distances: 1->2 = 3, 2->3 = 2, 1->3 = 5 via vertex 2.

Each k either ignores the new vertex or routes through it - that binary choice is the whole recurrence in action.

The trace shows the DP table improving exactly when a useful intermediate appears.

6 Compare Floyd Warshall with Dijkstra.

IMPORTANT

Scope: Floyd Warshall solves all-pairs shortest paths in one run; Dijkstra solves single-source, so it must be run V times for all pairs.

Weights: Floyd Warshall tolerates negative edge weights; Dijkstra requires non-negative weights.

Time: Floyd Warshall is O(V^3); one Dijkstra is O(E log V), so V runs cost O(V * E log V).

On dense graphs, V runs of Dijkstra approach O(V^3 log V), making Floyd Warshall competitive or better.

Code simplicity: Floyd Warshall is three nested loops; Dijkstra needs a priority queue plus settled/relaxation bookkeeping.

Memory: Floyd Warshall keeps a V x V table, O(V^2); Dijkstra keeps O(V) distances per run.

Rule of thumb: all pairs on a dense graph, or any negative edges - Floyd Warshall; single source with non-negative weights - Dijkstra.

They answer different questions, so the exam tests whether you match the algorithm to the problem.

7 Explain BFS with an example. What is its complexity?

REPEATED (Oct 2021, Feb 2025)

Breadth First Search explores a graph level by level: all neighbours of the source first, then neighbours of neighbours, using a QUEUE.

Pseudocode:

```
queue = [source]; visited[source] = True
while queue:
u = queue.pop(0)
for v in neighbours(u):
if not visited[v]: visited[v] = True; queue.append(v)
```

Trace: S connected to A and B; A connected to C.

Queue starts [S].

```
Pop S, enqueue A, B -> queue [A, B]. Pop A, enqueue C -> [B, C]. Pop B ->
[C]. Pop C -> [].
```

Visit order: S, A, B, C - level 0, then level 1, then level 2.

Complexity: each vertex is enqueued once and each edge examined once, giving O(V + E) with an adjacency list.

The queue is the key: first-in-first-out order is what enforces the level-by-level discipline.

BFS is the natural way to explore outward evenly, which is why it finds shortest paths in unweighted graphs.

8 Why does BFS give the shortest path in unweighted graphs?

IMPORTANT

In an unweighted graph, the shortest path means the fewest edges.

The queue processes vertices strictly in order of their distance from the source: all distance-1 vertices before any distance-2 vertex, and so on.

Therefore the first time BFS reaches a vertex, it has arrived via the smallest possible number of edges - no shorter route can exist.

Recording each vertex's parent at discovery time traces one such shortest path back to the source.

With weighted edges this breaks down, because a two-edge path can be shorter than a one-edge path; Dijkstra is needed there.

FIFO order plus unit weights equals shortest paths - remove either ingredient and the guarantee vanishes.

9 What are the applications of BFS?

IMPORTANT

Shortest path in unweighted graphs: fewest hops between two nodes, e.g. degrees of separation in a social network.

Connected components: one BFS from each unvisited vertex labels every component of the graph.

Bipartiteness test: colour levels alternately; an edge joining two same-colour vertices proves the graph is not bipartite.

Web crawlers: expand outward from seed pages level by level, which is BFS over the link graph.

Shortest path in mazes and grids: each step costs one, so BFS finds the quickest route.

Peer-to-peer and broadcast networks use BFS-like flooding to reach all nodes evenly.

Whenever fewest steps or level order matters, BFS is the tool.

10 Explain DFS with an example. What is its complexity?

REPEATED (Oct 2021, Feb 2025)

Depth First Search explores a graph by going as deep as possible along each branch before backtracking, using recursion as an implicit stack.

Pseudocode:

```
dfs(u):
visited[u] = True
for v in neighbours(u):
if not visited[v]: dfs(v)
```

Trace: edges S-A, S-B, A-C, starting at S.

```
Visit S; first neighbour A: visit A; A's neighbour C: visit C. C has no
unvisited neighbours: backtrack to A, then to S.
Next neighbour B: visit B. Done. Visit order: S, A, C, B.
```

Complexity: each vertex is visited once and each edge examined once, giving O(V + E) with an adjacency list.

The recursion stack records the current path, which is why DFS is natural for backtracking-style problems.

DFS trades BFS's level order for depth, which unlocks cycle detection and topological sorting.

11 Compare BFS and DFS.

REPEATED (Oct 2021, Feb 2025)

Data structure: BFS uses a queue (FIFO); DFS uses a stack or recursion (LIFO).

Order: BFS visits level by level outward; DFS dives deep down one branch before backtracking.

Shortest paths: BFS finds fewest-edge paths in unweighted graphs; DFS gives no such guarantee.

Memory: BFS stores a whole frontier level, O(V) in the worst case; DFS stores one path, O(depth), usually smaller.

Applications: BFS for shortest paths, components and bipartiteness; DFS for cycle detection, topological sort and backtracking search.

Both run in O(V + E) on adjacency lists and visit every reachable vertex exactly once.

Same complexity, opposite exploration order - the exam loves this contrast.

12 What are discovery and finish times in DFS?

IMPORTANT

DFS stamps every vertex twice: discovery time when it is first visited, finish time when all its neighbours are done.

Colour view: white means undiscovered, gray means discovered but still on the recursion stack, black means finished.

Discovery time marks entry, when the vertex turns gray; finish time marks exit, when it turns black.

The intervals nest properly: if v is discovered while u is gray, v's interval sits inside u's, proving v is a descendant of u in the DFS tree.

These times classify edges in directed graphs: a back edge to a gray ancestor reveals a cycle.

Sorting vertices by decreasing finish time yields a topological order of a DAG.

The two timestamps turn a simple walk into a full structural analysis of the graph.

13 How do you find connected components of a graph?

IMPORTANT

A connected component is a maximal group of vertices all reachable from each other; maximal means no vertex can be added to it.

Algorithm: keep a visited array and a counter starting at 0.

```
for each vertex v:
if not visited[v]:
run BFS or DFS from v # marks exactly its component
count += 1
```

Each launch explores precisely one component, because the search cannot cross over to unreachable vertices.

When the loop ends, count equals the number of components and every vertex is labelled with its component.

Complexity is O(V + E), since each vertex and edge is processed once across all launches.

Counting launches of a graph search is the standard components algorithm.

14 How do you test whether a graph is connected?

IMPORTANT

A graph is connected if it has exactly one component, i.e. every vertex is reachable from every other vertex.

Run a single BFS or DFS from any vertex, say vertex 0, and count how many vertices get visited.

If the count equals V, all vertices were reached and the graph is connected.

If fewer than V were visited, some vertices are unreachable and the graph is disconnected.

This is the components algorithm stopped after the first launch, still costing O(V + E).

One search, one count, one answer.

15 Explain the types of edges in DFS of a directed graph.

IMPORTANT

DFS of a directed graph classifies every edge by the state of its target vertex at the moment it is explored.

Tree edge: points to a white (undiscovered) vertex; it becomes part of the DFS tree.

Back edge: points to a gray vertex, i.e. an ancestor on the current recursion stack; its presence proves a directed cycle exists.

Forward edge: points to a black descendant, skipping down levels of the DFS tree.

Cross edge: points to a black vertex that is neither ancestor nor descendant, typically sitting in another branch or tree.

In undirected graphs only tree and back edges exist, which is why cycle detection there is simpler.

Edge colours are read off the discovery and finish times, turning DFS into a cycle detector.

16 How does DFS detect a cycle in a directed graph?

IMPORTANT

A directed graph has a cycle if and only if DFS encounters a back edge - an edge to a vertex currently on the recursion stack.

Colour scheme: white means unvisited, gray means on the current DFS path, black means fully explored.

Pseudocode:

```
dfs(u):
colour[u] = gray
```

```
for v in neighbours(u):
if colour[v] == gray: cycle found -> return True
if colour[v] == white and dfs(v): return True
colour[u] = black; return False
```

Trace: edges A->B, B->C, C->A. dfs(A) grays A; dfs(B) grays B; dfs(C) sees neighbour A which is gray - back edge, cycle confirmed.

Without the gray distinction, the edge C->A would look like an ordinary visited edge and the cycle would be missed.

Complexity is O(V + E).

For undirected graphs the rule simplifies to: a visited neighbour that is not the parent means a cycle.

Gray means my ancestor - an edge back to one is a cycle, always.

17 What is topological sorting? How is it done with DFS?

IMPORTANT

A topological ordering of a DAG lists vertices so that every directed edge u->v has u before v; it models task scheduling with prerequisites.

It exists only for directed acyclic graphs - a cycle makes ordering impossible, since each vertex would have to precede itself.

DFS method: run DFS over the whole graph; whenever a vertex finishes (turns black), push it onto a stack.

```
After DFS completes, pop the stack: vertices emerge in decreasing order of finish time.
```

Why it works: for any edge u->v, v finishes before u (v is explored from u), so u is pushed later and popped earlier.

Example: edges A->B, A->C, B->D, C->D.

Valid orders include A, B, C, D and A, C, B, D.

Complexity is O(V + E).

Finish time reversed is the one-line recipe for topological sort.

18 Explain the backtracking technique with an example.

REPEATED (Feb 2025, Oct 2025 H)

Backtracking solves constraint problems by building candidates one choice at a time, abandoning a partial candidate the moment it cannot lead to a valid answer.

Think of a maze: walk forward; at a dead end, walk back to the last junction and try another turn - the walking back is the backtrack.

The choices form a state space tree: each node is a partial answer and its children add one more choice.

A bounding function prunes branches that cannot succeed, so huge parts of the tree are never explored.

Example: N-Queens places queens row by row; the moment a queen is attacked, that placement is undone and the next column is tried.

Pseudocode shape:

```
solve(partial):
if partial is complete: record it; return
for each legal next choice:
extend partial; solve(partial); undo the extension # backtrack
```

Backtracking is organised trial and error: try, test early, undo fast.

19 What is a state space tree? What is pruning?

IMPORTANT

The state space tree is the tree of all partial candidates: the root is the empty answer, and each child adds one more choice.

A complete root-to-leaf path is one full candidate; internal nodes are the prefixes tried along the way.

Pruning means cutting off a subtree without exploring it, using a bounding function that proves no solution lies inside.

A node is called promising if the partial answer could still extend to a solution; only promising nodes are expanded.

Example: in sum of subsets with target 5, any branch whose partial sum already exceeds 5 is pruned immediately.

Good pruning is what separates backtracking from blind brute force - it can shrink an exponential search to something practical.

The tree organises the search; pruning keeps it affordable.

20 Differentiate backtracking from brute force.

IMPORTANT

Brute force generates every complete candidate first and only then tests whether it satisfies the constraints.

Backtracking tests partial candidates during construction and abandons a branch as soon as it is doomed.

Example: 4-queens by brute force would test all 4^4 = 256 full placements; backtracking kills most branches after one or two rows.

Both are exponential in the worst case, since a problem may genuinely need the whole tree explored.

But in practice backtracking with strong pruning runs far faster, because dead branches die young.

Memory also favours backtracking: it needs only the current path, O(depth), while brute force may hold many candidates.

Brute force checks everything at the end; backtracking checks early and quits early.

21 Explain the N-Queens problem and its backtracking solution.

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

The N-Queens problem asks for placements of N queens on an N x N board so that no two queens attack each other.

Attack rule: no two queens may share a row, a column, or a diagonal.

Backtracking strategy: place queens row by row from top to bottom, so rows are automatically distinct.

Pseudocode:

```
place(row):
if row == N: record the board; return
for col in 0..N-1:
if square (row, col) is safe:
put queen; place(row+1); remove queen # backtrack
```

Safety test against earlier queens at (r, c): col != c for the column, and abs(row - r) != abs(col - c) for the diagonals.

If no column is safe in a row, the recursion returns and the previous queen moves to its next column - that is the backtrack step.

Results: 4-queens has 2 solutions and 8-queens has 92; the worst case is exponential but

pruning makes it practical.

One queen per row plus the attack test turns an impossible search into a guided one.

22 How do you test whether a square is under attack?

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

Since queens are placed top to bottom, only earlier rows can threaten the new square.

Column test: the new square's column must differ from every earlier queen's column.

Diagonal test: for each earlier queen at (r, c), the new square (row, col) is attacked if abs(row - r) == abs(col - c).

The absolute-difference trick works because diagonals have slope 1: equal row-gap and column-gap means the same diagonal.

If the column test and both diagonal directions pass, the square is safe to use.

Example: queens at (0,1) and (1,3); square (2,0): columns differ; |2-0|=2 vs |0-1|=1 fine; |2-1|=1 vs |0-3|=3 fine - safe.

Three cheap arithmetic tests replace any board scanning.

23 Draw the state space tree concept for 4-queens.

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

Level 0 is the root: the empty board.

Each deeper level adds one queen in the next row.

Level 1 has 4 children: the first-row queen in columns 1, 2, 3, 4.

Level 2: under each level-1 node, only columns not attacked by the first queen become children - most branches are already pruned.

For example, a queen at (1,1) attacks column 1 and the diagonal through column 2 of row 2, leaving few children.

Levels 3 and 4 continue the same way; branches die whenever a row has no safe column, triggering a backtrack.

Only the paths that place all 4 queens safely reach level 4 - exactly 2 leaves, which are the 2 solutions.

The tree visualises pruning: the vast majority of the 4^4 theoretical nodes never get created.

24 Explain the sum of subsets problem with backtracking.

IMPORTANT

Problem: given n numbers (weights) and a target m, find all subsets whose elements sum to exactly m.

Sort the weights first; then decide for each weight in order: include it or skip it.

Pseudocode with pruning:

```
backtrack(k, cur, rem): # k = next index, cur = sum so far, rem = sum of
weights k..n
if cur == m: record the subset; return
if k > n or cur > m: return # overshoot: prune
if cur + rem < m: return # cannot reach m: prune
backtrack(k+1, cur + w[k], rem - w[k]) # include w[k]
backtrack(k+1, cur, rem - w[k]) # skip w[k]
```

Trace: weights [2, 3, 5], target 5.

Include 2, include 3: cur = 5, record {2,3}.

Backtrack, skip 3: cur = 2, include 5 gives cur = 7 > 5, prune.

The remaining branches yield {5}.

The two bounding tests - overshoot and cannot-reach - kill dead branches early.

Complexity is O(2^n) worst case, but sorting plus bounding prunes most of the tree in practice.

Include-or-skip plus two bounds is the complete backtracking solution.

25 What bounding functions prune the sum-of-subsets tree?

IMPORTANT

Bounding functions prove a branch is hopeless so it can be pruned without exploration.

Upper bound (overshoot): if the current partial sum already exceeds m, no extension can fix it - prune the branch.

Lower bound (unreachable): if the current sum plus the sum of ALL remaining weights is still below m, the target is unreachable - prune.

In the fixed-tuple formulation these become precise tests at each node k:

```
include w[k+1] only if cur + w[k+1] <= m # fits without overshoot
skip w[k+1] only if cur + remaining - w[k+1] >= m # target still reachable
```

Both bounds need the weights sorted and the remaining sums precomputed.

One bound cuts branches that are too big, the other cuts branches that are too small.

26 What is the time complexity of sum of subsets?

IMPORTANT

In the worst case the algorithm explores the include-or-skip choice for every weight: 2^n subsets, so O(2^n) time.

Pruning with sorted weights and the two bounding functions cuts typical searches dramatically, but the worst case stands.

The problem is NP-complete: no known algorithm solves all instances in polynomial time.

Space is O(n) for the recursion stack plus the current subset - the tree itself is never stored.

This exponential nature is exactly why backtracking with pruning, rather than brute force, is taught for this problem.

Standard exam answer: O(2^n) worst-case time, NP-complete, O(n) space.

27 Explain the Hamiltonian circuit problem and its backtracking solution.

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

A Hamiltonian circuit is a cycle that visits every vertex exactly once and returns to the start vertex.

Backtracking builds the path vertex by vertex; each new vertex must be adjacent to the last one and not visited before.

Pseudocode:

```
path = [start]
extend():
if len(path) == n:
if edge from path[-1] to start exists: circuit found
return
for each neighbour v of path[-1]:
if v not in path: append v; extend(); remove v # backtrack
```

Trace on square A-B-C-D-A: path grows A -> B -> C -> D; D connects back to A, so A-B-C-D-A is a Hamiltonian circuit.

If a dead end is hit (no unvisited neighbour and the path is incomplete), the last vertex is removed and the next neighbour is tried.

Worst case is O(n!) but adjacency pruning - only real neighbours are ever tried - keeps small graphs practical.

The problem is NP-complete, so exponential search is expected rather than a flaw.

Grow the path, demand adjacency, backtrack on dead ends, close the cycle at the end.

28 Differentiate Hamiltonian circuit from Euler circuit.

IMPORTANT

Hamiltonian circuit: visits every VERTEX exactly once, then returns to the start; edges may be skipped.

Euler circuit: uses every EDGE exactly once and returns to the start; vertices may repeat.

Euler has a simple test: a connected graph has an Euler circuit if and only if every vertex has even degree.

Hamiltonian has no such easy test; it is NP-complete and needs backtracking search.

Example: the square A-B-C-D-A is both Hamiltonian (visits A, B, C, D) and Eulerian (uses all 4 edges; every degree is 2).

Add a diagonal A-C: degrees become A:3, C:3 - odd, so no Euler circuit exists; but A-B-C-D-A remains Hamiltonian.

Euler cares about edges and has a degree test; Hamilton cares about vertices and has no shortcut.

29 What pruning is possible in Hamiltonian circuit search?

IMPORTANT

Only neighbours of the current last vertex are ever tried - non-adjacent vertices are pruned by construction.

Vertices already in the path are skipped, which enforces the exactly-once rule automatically.

If the unvisited vertices cannot form a connected chain back to the start, the branch is abandoned early.

Fixing the start vertex avoids rediscovering the same circuit n times in rotated form.

A degree check helps: a vertex (other than the start) with fewer than 2 unused incident edges can never belong to a circuit.

Despite all this, the worst case stays exponential - pruning shrinks the constant, not the complexity class.

Adjacency plus visited-set plus start-fixing is the standard pruning package for this problem.